FRI 프로토콜 상세 분석

개요

FRI(Fast Reed-Solomon Interactive Oracle Proof)는 Aurora 프로토콜의 핵심 구성 요소로, 주어진 함수가 저차 다항식에 가까운지 효율적으로 검증하는 프로토콜입니다. 이 문서는 FRI의 단계별 동작 과정, 폴딩(folding) 기법, 쿼리 단계의 일관성 검증 방법, 그리고 코셋 해싱 최적화에 대해 상세히 분석합니다.

FRI의 단계별 동작 과정

1. FRI 프로토콜의 전체 구조

FRI는 다음과 같은 단계로 구성됩니다:

```
초기화 → 커밋 단계 → 쿼리 단계 → 검증
↓ ↓ ↓ ↓
도메인 설정 폴딩 과정 일관성 검증 최종 판정
```

2. 초기화 단계

2.1 도메인 계층 구조 설정

```
template<typename FieldT>
class FRI_protocol : public multi_LDT_base<FieldT> {
std::vector<field_subset<FieldT> > domains_;
```

```
std::vector<field_subset<FieldT> > localizer_domains_;

std::vector<localizer_polynomial<FieldT> > localizer_polynomials_;

};
```

도메인 계산 과정:

```
void  compute_domains() {

// 초기 도메인부터 시작하여 각 라운드마다 크기 축소

for (size_t i = 0; i < num_reductions_; i++) {

size_t reduction_factor = 1 << localization_parameters_[i];

domains_[i+1] = domains_[i].get_subset_of_order(

domains_[i].num_elements() / reduction_factor);

}

}
```

2.2 지역화 다항식 (Localizer Polynomial) 설정

곱셈 도메인의 경우:

```
q(x) = x^{2^η} (η는 지역화 매개변수)
```

덧셈 도메인의 경우:

```
q(x) = 아핀 시프트 없는 L₀^(i)의 소멸 다항식
```

3. 커밋 단계 (Interactive Phase)

3.1 폴딩 과정의 수학적 원리

FRI의 핵심인 폴딩 과정은 다음과 같이 동작합니다:

기본 폴딩 공식:

```
f(x) = f_even(x²) + x · f_odd(x²)
```

여기서:

- `f_even(x²)` : f(x)의 짝수 차수 항들
- `f_odd(x²)` : f(x)의 홀수 차수 항들

일반화된 폴딩:

```
f_{i+1}(y) = Σ_{j=0}^{2^η-1} f_i(g^j · y^{1/2^η}) · L_j(α_i)
```

여기서:

- `η` : 지역화 매개변수
- `α_i` : i번째 라운드의 검증자 챌린지
- `L_j` : 라그랑주 기저 다항식

3.2 구현에서의 폴딩 과정

```
template<typename FieldT>
std::shared_ptr<std::vector<FieldT>> evaluate_next_f_i_over_entire_domain(
const std::shared_ptr<std::vector<FieldT>> &f_i_evals,
const field_subset<FieldT> &f_i_domain,
const  size_t  coset_size,
```

```
const FieldT x_i)
```

단계별 동작:

1. **코셋 분할**: f_i 도메인을 지역화 다항식에 의해 코셋들로 분할
2. **라그랑주 보간**: 각 코셋에서 다항식 P를 보간
3. **점 평가**: P(x_i)를 계산하여 f_{i+1}의 값 결정

3.3 덧셈 도메인에서의 최적화

```
template<typename FieldT>
std::shared_ptr<std::vector<FieldT>> additive_evaluate_next_f_i_over_entire_doma
const std::shared_ptr<std::vector<FieldT>> &f_i_evals,
const field_subset<FieldT> &f_i_domain,
const  size_t  coset_size,
const FieldT x_i)
```

최적화 기법:

- **소멸 다항식 재사용**: 코셋 간 공통 부분 활용
- **배치 역원 계산**: 여러 역원을 한 번에 계산
- **캐시 효율성**: 메모리 접근 패턴 최적화

3.4 곱셈 도메인에서의 최적화

```
template<typename FieldT>
std::shared_ptr<std::vector<FieldT>> multiplicative_evaluate_next_f_i_over_entir
const std::shared_ptr<std::vector<FieldT>> &f_i_evals,
```

```
const field_subset<FieldT> &f_i_domain,
const  size_t  coset_size,
const FieldT x_i)
```

라그랑주 계수 최적화:

```
L_k = (vp_coset(x) * z_k) / (x - v[k])
```

여기서:

- `z_k = g^k / (|coset| * h^{|coset|-1})`
- `vp_coset(x) = x^{|coset|} - h^{|coset|}`

배치 처리 최적화:

- 모든 코셋에 대해 단일 배치 역원 계산
- `xg^{-k}` 값들을 사전 계산하여 재사용
- 코셋별 상수 `vp_coset(x) / h^{|coset|-1}` 최적화

폴딩(Folding) 기법 상세 분석

1. 폴딩의 수학적 기초

1.1 차수 감소 원리

다항식 f(x)가 차수 d를 가질 때:

```
f(x) = Σ_{i=0}^d a_i x^i = Σ_{i=0}^{⌊d/2⌋} a_{2i} x^{2i} + x Σ_{i=0}^{⌊(d-1)/2⌋} a
```

이를 다시 쓰면:

```
f(x) = f_even(x²) + x · f_odd(x²)
```

여기서 f_even과 f_odd는 각각 최대 차수 ⌊d/2⌋를 가집니다.

1.2 일반화된 폴딩

지역화 매개변수 η에 대해:

```
f_{i+1}(y) = Σ_{j=0}^{2^η-1} f_i(ω^j · y^{1/2^η}) · L_j(α)
```

여기서:

- `ω` : 2^η차 단위근
- `L_j` : j번째 라그랑주 기저 다항식
- `α` : 검증자의 랜덤 챌린지

2. 폴딩의 보안성 분석

2.1 사운드니스 보장

핵심 정리: f가 차수 d의 다항식이면, 폴딩된 f'도 차수 ≤ d/2^η의 다항식입니다.

증명 스케치:

1. f가 실제로 차수 d라면, 폴딩 과정에서 차수가 정확히 감소
2. f가 차수 d가 아니라면, 높은 확률로 폴딩 후에도 높은 차수 유지
3. 재귀적 적용으로 최종적으로 저차가 아닌 함수 탐지

2.2 완전성 보장

정직한 프로버의 경우:

- 실제 저차 다항식에 대해서는 모든 폴딩 단계에서 일관성 유지
- 최종 다항식이 예상된 차수를 가짐
- 검증자의 모든 쿼리에 올바른 응답 제공

3. 구현에서의 폴딩 최적화

3.1 메모리 효율성

```
std::shared_ptr<std::vector<FieldT>> next_f_i = std::make_shared<std::vector<Fie

next_f_i->reserve(num_cosets);
```

최적화 전략:

- 스마트 포인터를 통한 메모리 관리
- 사전 할당을 통한 재할당 방지
- 코셋별 순차 처리로 캐시 효율성 향상

3.2 계산 복잡도 최적화

배치 역원 계산:

```
const std::vector<FieldT> lagrange_coefficients =

batch_inverse_and_mul(shifted_coset_elements, k);
```

효과:

- 개별 역원 계산: O(n log n) (n개의 역원)
- 배치 역원 계산: O(n) + O(log n) = O(n)

쿼리 단계에서의 일관성 검증 방법

1. 쿼리 단계의 전체 구조

```
struct FRI_query_set {
size_t interaction_index_;
size_t LDT_index_;
random_query_position_handle s0_position_handle_;
query_handle_set f_at_s_coset_query_handles_;
};
```

2. 쿼리 위치 계산

2.1 초기 쿼리 위치 선택

검증자가 랜덤하게 선택한 위치 s_0에서 시작하여, 각 폴딩 라운드에서 대응되는 위치들을 계산합니다.

```
template<typename FieldT>
std::vector<query_position_handle> calculate_next_coset_query_positions(
iop_protocol<FieldT> &IOP,
const query_position_handle &non_localized_query_handle,
const field_subset<FieldT> &non_localized_domain,
const field_subset<FieldT> &localized_domain,
const  size_t  prev_localization_parameter,
const  size_t  cur_localization_parameter);
```

2.2 코셋 내 위치 매핑

곱셈 도메인의 경우:

```
s_{i+1} = s_i^{2^η} (η는 지역화 매개변수)
```

덧셈 도메인의 경우:

```
s_{i+1} = 선형 변환을 통한 매핑
```

3. 일관성 검증 과정

3.1 폴딩 일관성 검증

각 라운드 i에서 다음을 확인합니다:

1. **코셋 값 수집**: s_i가 속한 코셋의 모든 값들을 수집
2. **라그랑주 보간**: 코셋 값들로부터 다항식 P 복원
3. **점 평가**: P(α_i) 계산
4. **일치성 확인**: P(α_i) = f_{i+1}(s_{i+1}) 검증

3.2 머클 증명 검증

```
bool  predicate_for_query_set(const FRI_query_set &Q) {
// 1. 머클 증명 검증
// 2. 폴딩 일관성 검증
// 3. 최종 다항식 차수 검증
}
```

검증 단계:

1. **인증 경로 확인**: 각 쿼리 값의 머클 증명 검증
2. **폴딩 계산**: 제공된 값들로 폴딩 결과 재계산
3. **일치성 확인**: 계산 결과와 다음 라운드 값 비교

4. 보안성 분석

4.1 사운드니스 오류 확률

증명된 사운드니스 (FRI 논문):

```
ϵ_query ≥ (1 - min(δ, (1-3ρ-2^η/√|L|)/4))^t
```

여기서:

- δ: 근접성 매개변수
- ρ: Reed-Solomon 코드 비율
- η: 지역화 매개변수
- |L|: 코드워드 도메인 크기
- t: 쿼리 반복 횟수

휴리스틱 사운드니스:

```
ϵ_query ≈ (1 - δ)^t
```

4.2 구현에서의 보안성 계산

```
// 증명된 사운드니스 계산
const long double term2 = (1 - 3*rate - (localization_coset_size / sqrt_codewo
```

```
this->effective_proximity_parameter_ = std::min<long double>(fractional_proximi
long double denominator = log2l(1 - this->effective_proximity_parameter_);
const long double query_repetitions = ceil(-1.0 * query_soundness_bits / denom
```

코셋 해싱(Coset Hashing) 최적화와 BCS 증명 시간 단축

1. 코셋 해싱의 개념

코셋 해싱은 FRI에서 머클 트리 구성을 최적화하는 기법입니다.

1.1 기본 아이디어

전통적 방식:

- 각 도메인 원소를 개별적으로 해싱
- 머클 트리 잎 노드 수 = 도메인 크기

코셋 해싱 방식:

- 코셋 단위로 해싱
- 머클 트리 잎 노드 수 = 코셋 개수

2. 구현에서의 코셋 해싱

2.1 코셋 기반 커밋먼트

```
// 코셋별로 값들을 그룹화하여 해싱
for (size_t coset_idx = 0; coset_idx < num_cosets; coset_idx++) {
std::vector<FieldT> coset_values;
```

```
for (size_t i = 0; i < coset_size; i++) {

coset_values.push_back(f_evals[coset_idx * coset_size + i]);

}

// 코셋 전체를 하나의 해시로 커밋

hash_values[coset_idx] = hash_function(coset_values);

}
```

2.2 쿼리 응답 최적화

전통적 방식:

- 각 쿼리마다 개별 머클 증명 제공
- 증명 크기: O(log |도메인|) × 쿼리 수

코셋 해싱 방식:

- 코셋 전체에 대한 단일 머클 증명
- 증명 크기: O(log |코셋 수|) + O(코셋 크기)

3. BCS 변환 시간 단축 효과

3.1 머클 트리 구성 시간

개선 전:

```
트리 높이: log₂ (도메인 크기)

해시 연산 수: O(도메인 크기)
```

개선 후:

```
트리 높이: log₂ (코셋 수) = log₂ (도메인 크기) - log₂ (코셋 크기)
```

```
해시 연산 수: O(코셋 수) = O(도메인 크기 / 코셋 크기)
```

3.2 증명 생성 시간

시간 복잡도 개선:

- 머클 증명 생성: O(log |도메인|) → O(log |코셋 수|)
- 해시 계산: O(|도메인|) → O(|도메인| / |코셋 크기|)

3.3 검증 시간 개선

검증자 관점:

- 머클 증명 검증: 더 짧은 인증 경로
- 코셋 내 일관성 검증: 추가 계산 필요하지만 전체적으로 효율적

4. 실제 성능 개선 효과

4.1 메모리 사용량

머클 트리 크기:

- 개선 전: O(도메인 크기)
- 개선 후: O(도메인 크기 / 코셋 크기)

4.2 통신 복잡도

증명 크기:

- 머클 증명 부분: 약 $\log_2$(코셋 크기)만큼 감소
- 코셋 데이터 부분: 코셋 크기만큼 증가
- 전체적으로는 감소 (특히 큰 도메인에서)

4.3 계산 복잡도

프로버:

- 해시 계산: 코셋 크기 배수만큼 감소
- 머클 트리 구성: 높이 감소로 인한 개선

검증자:

- 머클 증명 검증: 더 짧은 경로로 빠른 검증
- 코셋 일관성 검증: 추가 오버헤드 있지만 전체적으로 효율적

결론

FRI 프로토콜은 Aurora의 핵심 구성 요소로서, 정교한 폴딩 기법과 효율적인 쿼리 검증 메커니즘을 통해 $O(\log^2 N)$ 크기의 투명한 zkSNARK를 가능하게 합니다. 특히 코셋 해싱과 같은 최적화 기법들은 이론적 우수성을 실용적 효율성으로 변환하는 데 중요한 역할을 합니다.

FRI의 재귀적 구조와 확률적 검증 방식은 대규모 계산에서도 효율적인 영지식 증명을 가능하게 하며, 이는 현대 STARK 시스템들의 기반 기술로 널리 활용되고 있습니다. 구현에서의 다양한 최적화 기법들은 FRI를 단순한 이론적 프로토콜에서 실용적인 암호학적 도구로 발전시키는 데 핵심적인 역할을 하고 있습니다.